ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

INFORMATION RETRIEVAL

From plain text to triplets

DE VINCI
RESEARCH
CENTER

Data (text) pre-processing

Data (text) pre-processing: From raw to structured text

1. Word (term) extraction/segmentation
   - easy for space-delimited languages (*eg.* English and French)
   - Not straightforward for *scriptio continua* languages (*eg.* Chinese)
     - *Using probability models (eg. HMM+Viterbi) or sequential models (eg. LSTM, Transformer, etc.)*
2. Stop-words removal
3. Stemming/lemmatization
4. Word representation:
   - Tokenization
   - Frequency counts and computing TF-IDF term weights.

ESILV

Stop-words removal

- Many of the most frequently used words in English are useless in IR and text mining – these words are called stop-words.
  - the, of, and, to, ….
  - Typically, about 400 to 500 such words
  - For an application, an additional domain specific stopwords list may be constructed
- Why do we need to remove stop-words?
  - Reduce indexing (or data) file size
    - Stop-words accounts 20-30% of total word counts.
  - Improve efficiency and effectiveness
    - Stop-words are not useful for searching or text mining
    - they may also confuse the retrieval system.

ESILV

Stemming

- Techniques used to find out the root/stem of a word. E.g.,
  - user
  - users
  - used
  - using

  engineering
  engineered
  engineer
- stem: use engineer

Usefulness:

- improving effectiveness of IR and text mining
  - matching similar words
  - Mainly improve recall
- reducing indexing size
  - combing words with same roots may reduce indexing size as much as 40-50%.

ESILV

Basic stemming methods

Using a set of rules. E.g.,

- remove ending
  - if a word ends with a consonant other than s, followed by an s, then delete s.
  - if a word ends in es, drop the s.
  - if a word ends in ing, delete the ing unless the remaining word consists only of one letter or of th.
  - If a word ends with ed, preceded by a consonant, delete the ed unless this leaves only a single letter.
  - …...
- transform words
  - if a word ends with “ies” but not “eies” or “aies” then “ies --> y.”

ESILV

Lemmatization (1)

- Grouping together the inflected forms of a word so they can be analysed as a single item, identified by the word's lemma
  - Similar to Stemming, but more precise, with the consideration of the context
    - "better" has "good" as its lemma. This link is missed by stemming, as it requires a dictionary look-up.
    - "walk" is the base form for the word "walking", this is matched in both stemming and lemmatization.
    - The word "meeting" can be either the base form of a **noun** or a form of a **verb** ("to meet") depending on the context.
- Algorithms
  - A trivial way: dictionary lookup
  - Rule-based system (based on **PoS tagging**)

Lemmatization (2): Algo. by PoS tagging

- Marking up a word in a text (corpus) as corresponding to a particular **P**art **o**f **S**peech, based on both its definition and its context.
- Such tagging helps finding the lemma of words
- Example, with more tags here
- How to find the PoS tagging? We will see later in the named entities recognition part

Tag	Description	Example
NN	Singular common noun	dog
PRP	Peronal Pronoun	He, she, it
VB	Base form of a verb	Run
VBD	Past tense verb	ran
JJ	Adjective	quick
RB	Adverb	quickly
DT	Determiner (eg., articles)	the, a
IN	Preposition	in, on, because
UH	Interjection	Oh, wow

ESILV

Example

- Original Text: The quick brown foxes are jumping over the lazy dogs. Lemmatized Text: the quick brown fox be jump over the lazy dog .

The → Determiner (DT)
quick → Adjective (JJ)
brown → Adjective (JJ)
fox → Noun (NN)
jumps → Verb (VBZ)
over → Preposition (IN)
the → Determiner (DT)
lazy → Adjective (JJ)
dog → Noun (NN)

ESILV

Lemmatization (3): implementation with WordNet Lemmatizer

- The Natural Language Toolkit (**NLTK**) and **spaCy** provides a class called `WordNetLemmatizer`, which is a thin wrapper around the WordNet corpus, based on *dictionary lookup*

```
# import these modules
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

print("rocks :", lemmatizer.lemmatize("rocks"))
print("corpora :", lemmatizer.lemmatize("corpora"))

# a denotes adjective in "pos"
print("better :", lemmatizer.lemmatize("better", pos="a"))
```

rocks : rock
corpora : corpus
better : good

pos*(str)*-the PoS tag*:*
"n" for nouns,
"v" for verbs,
"a" for adjectives,
"r" for adverbs and
"s" for satellite adjectives.

ESILV

Tokenization

- The process of breaking a stream of text into smaller meaningful elements called **tokens**. The tokens can be words, phrases, symbols, or even sub-words depending on the application

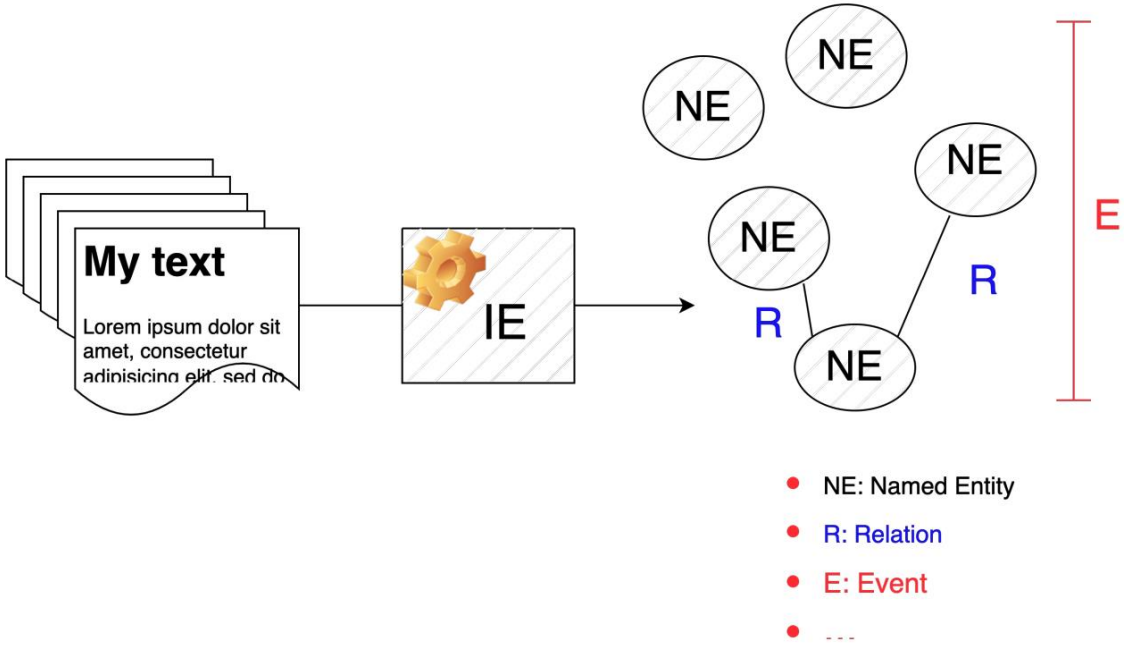
ESILV

Information Retrieval

ESILV

Information Extraction (IE) in practice: NE

- **Named entity**: A lexical unit which refers to a real-word object classified within a category: people, organization, location, money, percent, etc
- **Named Entity Recognition (NER)**: a salient sub-task of IE. Involves three steps: recognition, categorization, normalization (if needed)



"Epidemie Ebola : 2013-2015

Cette épidémie, commencée en décembre 2013, est parfois qualifiée d'atypique », parce que non maîtrisée. En juillet 2014, elle évolue de manière préoccupante en Guinée, au Liberia et en Sierra Leone. Le 20 août 2014, 844 décès étaient officiellement confirmés comme dus au virus69. Une flambée se déclare dans le district de Boende."

Disease

Location

Date

ESILV

Information Extraction (IE) in practice: Relation

- Relations:
  - Facts generally involving named (named) entities
  - Facts as n-ary relations between entities
  - Literature focuses on binary relations, converted into knowledge triples:

    `(subject, predicate, object)`

Plusieurs manifestants sont désignés comme porte-parole par des Gilets jaunes ou sont mis en avant par les médias. Au début du mouvement, les principales personnalités médiatisées "sont Éric Drouet, qui a lancé sur Facebook l'appel au rassemblement du 17 novembre 2018, Priscillia Ludosky, à l'origine de la pétition en ligne de mai 2018 appelant à la baisse des prix du carburant, Jacline Mouraud, dont la vidéo à l'adresse d'Emmanuel Macron est devenue virale

subject	predicate	object
Eric Drouet	personnalité	Gilets Jaunes
Priscillia Ludosky	personnalité	Gilets Jaunes
Jacline Mouraud	personnalité	Gilets Jaunes
Eric Drouet	Origine	Rassemblement
Priscillia Ludosky	Origine	Petition

ESILV

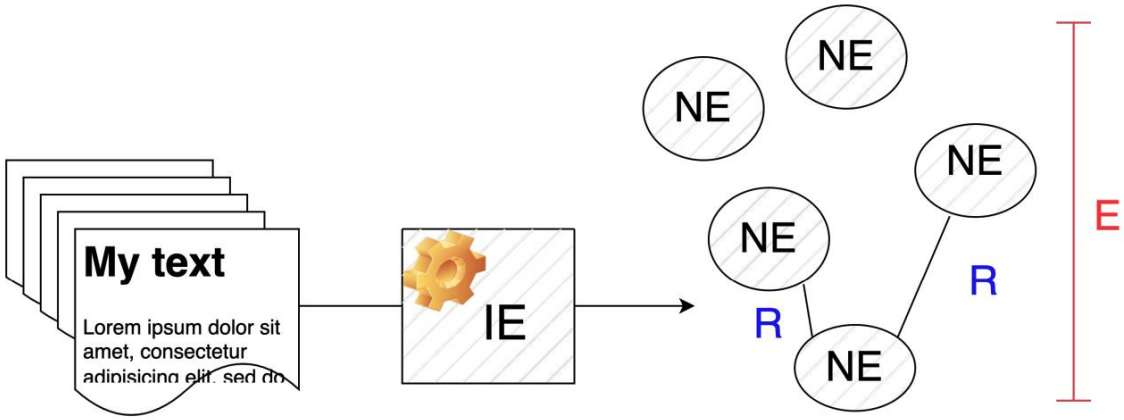
Information Extraction (IE) in practice: Events

- There is no standard definition for Event, it is a specific occurrence triggered at a place and time
- An award event ”Cedric Viliani from France, won the Fields Medal in 2010”
  - Entities: objects involved (Cedric Villiani, Fields Medal)
  - Event type: nature of the event (wining)
  - Event trigger: a verb or noun as the core unit of the evnt (won)
  - Event arguments: attributes of the event.(2010)
  - Argument role: relationship between the event and the argument eg. time, date, place, (2010: date)

ESILV

Extracting Named Entities: NE recognition (NER)

- The task of NER



Elisabeth II, the queen of Queen of the United Kingdom died on September 2022 at Balmoral → NER →

```
< Elisabeth, II; Person>
< United, Kingdom; Location>
< September, 2022; Location>
<Balmoral; Location>
```

- NE: Named Entity
- R: Relation
- E: Event
- ...

- How to do NER?

Approaches for NER

- **Rule-based**
  - Manual/hand-written patterns
- Unsupervised learning
  - Mostly clustering techniques
  - Discriminative models
- Feature-based supervised learning
  - Naive Bayes, Hidden Marko Models (HMM), Conditional Random Field (CRF), Maximum-Entropy Markov Models (MEMMs)
- (Deep) Neural - based learning
  - Distributed representation of inputs, deep encoding of context, tags
- Transformer based learning

ESILV

Approaches for NER: Rule-based methods

- A rule describes a pattern for a NE category (Org, Per, etc.)
  - Recognizes the applicability of the rule:
    - Matches the instances of the category based on regularities
  - If the context is applicable then:
    - Bound the NE: <Begin /NE….End/NE>
    - Assign the NE to the category
- Features used for NER
  - Lexical features
    - Position, punctuation, spelling, ...
  - Syntaxic features
    - Prefix, suffix, grammatical tag, negation, ..
  - Semantic features
    - Polarity (positive, negative, neutral), relation with word contexts, ...

Example

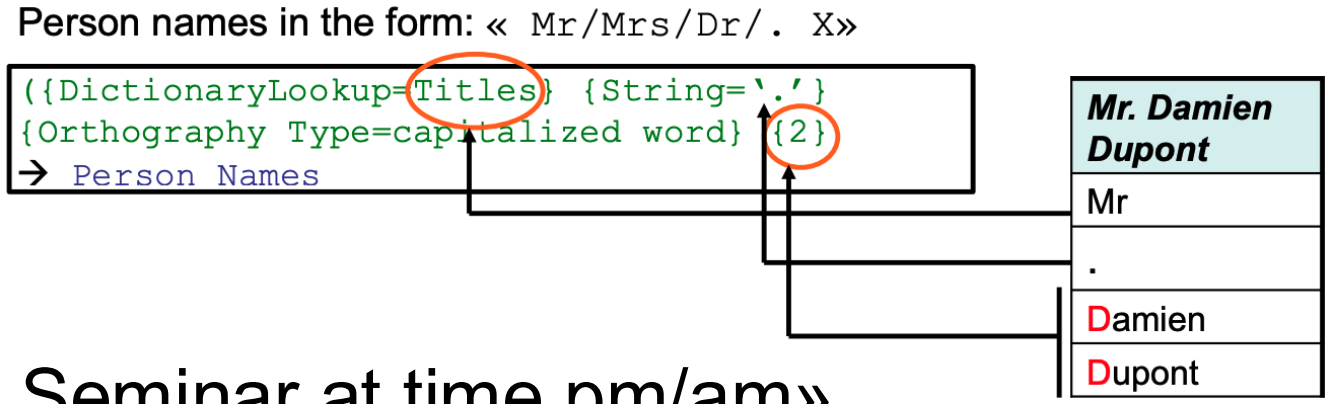
Date	
Year	(aaaa)
Month	(m-nom dans {Janvier…Décembre})
Month -no	(m-number=integer and 0<m-number<=12)
Day	(j-name dans {Lundi…Dimanche})
Day-no	(j-number=integer and 1<m-number<=long(mois));

ESILV

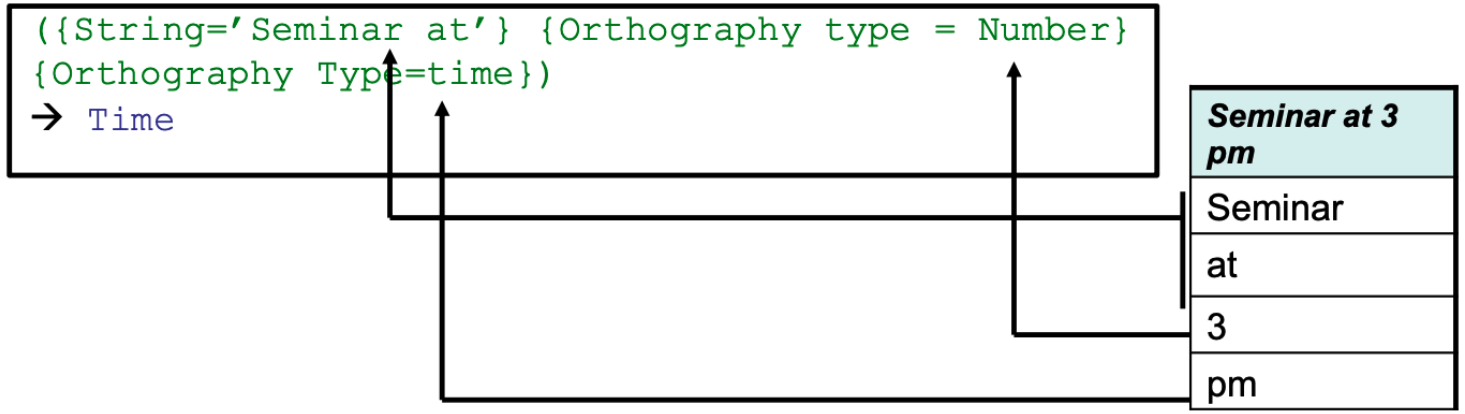
Rule-based methods: Examples

General Architecture for Text Engineering (GATE)

- Person names in the form: « Mr/Mrs/Dr/. X»



- Seminar schedule: « Seminar at time pm/am»



Rule-based methods: Examples

- Some NE requires a pair of rules: `Begin` and `End` for tagging and bounding
  - Expression: « To appear ‘Le monde ’ Vol. 5 »
- Some patterns allow bounding a set of NE (eg. rich tabular forms)
  - Recognize room number and renting price

```
({String='to appear in'})jstart ({Orthographe
Type=Capitalized word} {2-5})
→ Insert <journal> after:jstart
```

```
({tag=<journal>}) jstart ({Orthographe
Type=word}+):jend {String='vol'})
→ Insert </journal> after:jend
```

```
({Ortography
Type=Digit}):Bedrooms({String='BR'})({}*)({String='$'
})({Ortography Type=Number}):Price
→ Number of Bedrooms=:Bedroom, Rent=:Price <journal>
```


ESILV

Rule-based methods: implementation

- Extract by Regular Expression (RE): More details
- Example:
  - Extract organization names

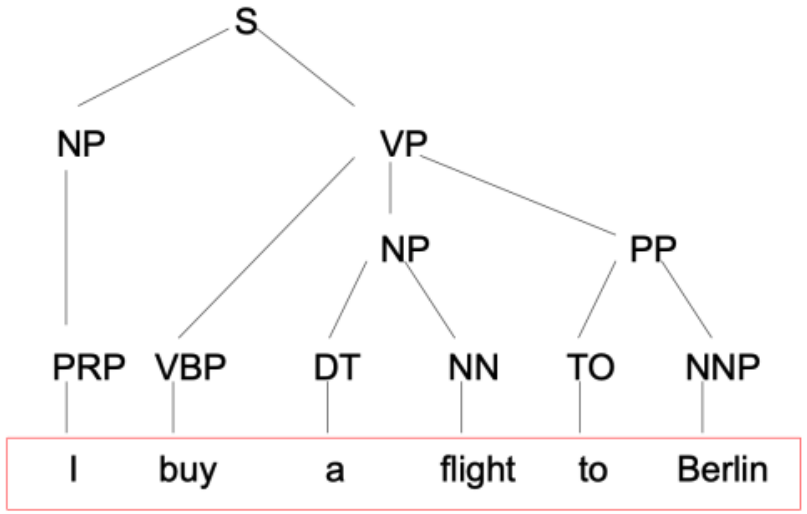
```
In [1]: import re

In [2]: example = "Department of Computer Science. Faculty of Science and En
   ...: gineering, ESILV, France"

In [3]: org = re.search(r"([AZ][^\s,.]+[.]?\s[(]?)*(Dept|Association|Departm
   ...: ent|University|Faculty)[^,\d]*?(?=,|\.|\d)", example).group(0)

In [4]: print(org)
Department of Computer Science
```

  - Extract e-mail addresses

```
In [12]: str='purple alice@google.com, blah monkey bob@abc.com blah dishwasher

In [13]: emails = re.findall(r'[\w\.-]+@[\w\.-]+', str)

In [14]: print(emails)
['alice@google.com', 'bob@abc.com']
```

Rule-based methods: Using NLP tags

- NLP might help designing hand-written rules
  - Uses NLP tags
    - Part Of Speech (POS) tagging
    - Syntactic parsing: mark words as nouns, verbs, preposition, etc.
    - Semantic word categories from an external resource (Yago, Freebase, WordNet, etc.)
  - Rule-based expressions with NLP
    - Which person holds what office in what organization: `[Pers], [office] of [Org]`
    - Where an organization is located: `[Org] in [Loc]`



Limitations of rule-based methods

- Historically (1987-1997) from Message Understanding events (MUC). Main tasks focused on extracting from news:
  - NER, Coreference resolution, Information Extraction
- Limitations of rule-based methods
  - Cost of the definition of hand-written rules: 1 rule per form ! Useful for limited "clear" patterns
  - Recall might be low in rich texts
  - Gradually moved to machine-learning based methods

ESILV

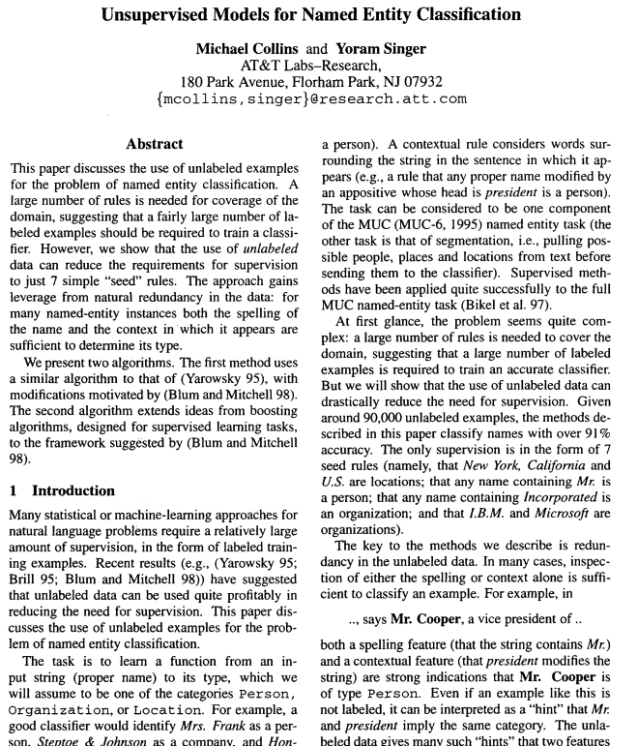
Approaches for NER

- Rule-based
  - Manual/hand-written patterns
- **Unsupervised learning**
  - Mostly clustering techniques
  - Discriminative models
- Feature-based supervised learning
  - Naive Bayes, Hidden Marko Models (HMM), Conditional Random Field (CRF), Maximum-Entropy Markov Models (MEMMs)
- (Deep) Neural - based learning
  - Distributed representation of inputs, deep encoding of context, tags

ESILV

NER-Unsupervised learning methods

- Key idea
  - No human-labeled text
- General pipeline
  - Select features to encode/represent input text
    - Manual/hand-written rules
  - Apply a clustering model to feature-based descriptors
    - The model uses a combination of features
    - The model assigns to tokens in the input text **automatically-generated labels**
- Generally, learns one set of rules per NE category



Unsupervised Models for Named Entity Classification

Michael Collins and **Yoram Singer**
AT&T Labs–Research,
180 Park Avenue, Florham Park, NJ 07932
{mcollins,singer}@research.att.com

Abstract

This paper discusses the use of unlabeled examples for the problem of named entity classification. A large number of rules is needed for coverage of the domain, suggesting that a fairly large number of labeled examples should be required to train a classifier. However, we show that the use of *unlabeled* data can reduce the requirements for supervision to just 7 simple "seed" rules. The approach gains leverage from natural redundancy in the data: for many named-entity instances both the spelling of the name and the context in which it appears are sufficient to determine its type.

We present two algorithms. The first method uses a similar algorithm to that of (Yarowsky 95), with modifications motivated by (Blum and Mitchell 98). The second algorithm extends ideas from boosting algorithms, designed for supervised learning tasks, to the framework suggested by (Blum and Mitchell 98).

1 Introduction

Many statistical or machine-learning approaches for natural language problems require a relatively large amount of supervision, in the form of labeled training examples. Recent results (e.g., (Yarowsky 95; Brill 95; Blum and Mitchell 98)) have suggested that unlabeled data can be used quite profitably in reducing the need for supervision. This paper discusses the use of unlabeled examples for the problem of named entity classification.

The task is to learn a function from an input string (proper name) to its type, which we will assume to be one of the categories `Person`, `Organization`, or `Location`. For example, a good classifier would identify *Mrs. Frank* as a person, *Steptoe & Johnson* as a company, and *Hon-*

a person). A contextual rule considers words surrounding the string in the sentence in which it appears (e.g., a rule that any proper name modified by an appositive whose head is *president* is a person). The task can be considered to be one component of the MUC (MUC-6, 1995) named entity task (the other task is that of segmentation, i.e., pulling possible people, places and locations from text before sending them to the classifier). Supervised methods have been applied quite successfully to the full MUC named-entity task (Bikel et al. 97).

At first glance, the problem seems quite complex: a large number of rules is needed to cover the domain, suggesting that a large number of labeled examples is required to train an accurate classifier. But we will show that the use of unlabeled data can drastically reduce the need for supervision. Given around 90,000 unlabeled examples, the methods described in this paper classify names with over 91% accuracy. The only supervision is in the form of 7 seed rules (namely, that *New York, California* and *U.S.* are locations; that any name containing *Mr.* is a person; that any name containing *Incorporated* is an organization; and that *I.B.M.* and *Microsoft* are organizations).

The key to the methods we describe is redundancy in the unlabeled data. In many cases, inspection of either the spelling or context alone is sufficient to classify an example. For example, in

.., says **Mr. Cooper**, a vice president of ..

both a spelling feature (that the string contains *Mr.*) and a contextual feature (that *president* modifies the string) are strong indications that **Mr. Cooper** is of type `Person`. Even if an example like this is not labeled, it can be interpreted as a "hint" that *Mr.* and *president* imply the same category. The unlabeled data gives many such "hints" that two features

M. Collins, Y. Singer; Unsupervised models for named entity classification EMNLP 1999

The Collins & Yoram method

- Key idea
  - High-level of redundancy in the data makes possible to automatically generate/learn "labelling rules" ie. assign a NE category (`Person, Location, Organization`) to a token. well known today as self-supervision
- Key notion: feature-based rule
  - Parse sentences/strings in the text: recognize capitals, grammatical positions (Noun, Verb, Preposition, modifiers, etc.
  - Feature-based predicate: indicates the properties of the string on a pair of **(spelling, context)** pair
    - Spelling: forms of the words within the string
    - Context: forms and roles of the words surrounding the sentence
  - Feature-based rule: feature-based predicate that automatically assigns to the string/sentence **an entity category**

ESILV

Sentence	Entities (Spelling/Context)	Features
But Robert Jordan, a partner at Steptoe & Johnson who took ...	Robert Jordan/partner	full-string=Robert_Jordan contains(Robert) contains(Jordan) context=partner context-type=appos
	Steptoe & Johnson/partner_at	full-string=Steptoe_&_Johnson contains(Steptoe) contains(&) contains(Johnson) nonalpha=& context=partner_at context-type=prep
By hiring a company like A.T.&T. ...	A.T.&T./company_like	full-string=A.T.&T. allcap2 nonalpha=..&. context=company_like context-type=prep
Hanson acquired Kidde Incorporated, parent of Kidde Credit, for ...	Kidde Incorporated/parent	full-string=Kidde_Incorporated contains(Kidde) contains(Incorporated) context=parent context-type=appos
	Kidde-Credit/parent_of	full-string=Kidde_Credit contains(Kidde) contains(Credit) context=parent_of context-type=prep

Feature-based descriptors of strings

```
full-string=New_York        →   Location
full-string=California      →   Location
full-string=U.S.            →   Location
contains(Mr.)               →   Person
contains(Incorporated)      →   Organization
full-string=Microsoft       →   Organization
full-string=I.B.M.          →   Organization
```

Feature-based EN labelling rules

The general algorithm of automatic generation of labeling rules

– Input: training **T** text, **R** a set of seed rules; **n**: number of rules generated at each iteration, **k**: number of labels

Begin

1- Automatically label the training texts **T** using **R**

2- Induce contextualized rules from labeled data

3- Select the most confident contextual rules (less than **n***k**)

4- Apply the contextual rules generated in 3 to automatically label text **T**

5- Select the most confident spelling rules (less than **n*k**)

6- n=n+5 ; if n<threshold

Add the n top selected spelling rules to R/spelling and contextual rules;

Return to step 1.

End

ESILV

- Advantages
  - Limited amount supervision/labels is initially needed
  - Possibility of using a wide range of supervised learning algorithms for self-generating new labeling rules (naive bayes, decision trees, bootstrapping)
- Limitations
  - Performance heavily depends on seed rules
  - Lack of generalizability, exhaustivity of rules: diversity of contexts lead to noisy rules
  - Requires the definition of seed hand-written rules ...inherently induces limitations of this method!

ESILV

Approaches for NER

- Rule-based
  - Manual/hand-written patterns
- Unsupervised learning
  - Mostly clustering techniques
  - Discriminative models
- **Feature-based supervised learning**
  - Naive Bayes, Hidden Marko Models (HMM), Conditional Random Field (CRF), Maximum-Entropy Markov Models (MEMMs)
- (Deep) Neural - based learning
  - Distributed representation of inputs, deep encoding of context, tags

ESILV

NER-Supervised learning methods

- Key idea
  - Automatically learn features-based models for classifying entities
  - A class is a category label
- General pipeline
  - Identify a set of target NE categories to classify
  - Design relevant features to encode/represent input text (either training or testing) for these NE categories. Build/code the extractors
  - Training
    - Dispose of human-labelled data (training collection): each token is labeled with its category/class
    - Train a text classification model (NB, CRF, HMM, etc.) to predict the entity label class from the data
  - Testing
    - Dispose of a testing collection (to be automatically labeled)
    - Run the trained classification model
    - Output the predicted entities labels/classes

ESILV

Feature Extraction (1)

- Lexical: capitalization, special characters, punctuation, ...
- Syntaxic: POS tags (*e.g.*, V, NN, *etc.*)
- Semantic features: semantic category is a resource,
- Discourse features: computed by using text fragments beyond the sentence.

x_1	x_2	x_3	x_4
Paul	habite (lives)	à (in)	Paris
pers	nul	nul	loc

: X word sequence $x_1, x_2, \ldots x_n$

: Y entity class among possible ones: Pers, Loc,Org, Time, etc.

Le	séminaire	commence	à	13H30	cet	après-midi
(The	seminar	starts	at)		(this	afternoon)
		nul		time		nul

ESILV

Feature Extraction (2): measure the features

- Build the feature-based descriptors based on feature functions: basic or composite functions, regular expressions, ...
- Features can have Boolean, numerical, range values, ...
- On the current word, on previous words, ? next words (context)

F1 (Y,X,i): feature value x_i with target label Y (trainig data)

Features

```
F1: First letter of xi is capital
F2: xi-1='.'
F3: xi contains special character
F4: length of xi
```

Paul	habite	à	Paris
pers	nul	nul	loc

```
F1(pers,X,1)=1   F1(nul,X,2)=0
F2(pers,X,1)=0   F2(nul,X,2)=0
F3(pers,X,1)=0   F3(nul,X,2)=0
F4(pers,X,1)=4   F4(nul,X,2)=6
```

Measure the features

F1 (Y,X,i): feature value x_i with target label Y (trainig data)

Features

```
F1: First letter of xi is capital
F2: xi-1 contains '.'
F3: xi contains special character
F4: length of xi
```

X	1	2	3	4
	M.	Albert	à	Rio
F1	1	1	0	1
F2	0	1	0	0
F3	1	0	0	0
F4	2	6	1	3

Feature vector 3 over X

Feature vector of X_1

Extract features with Python

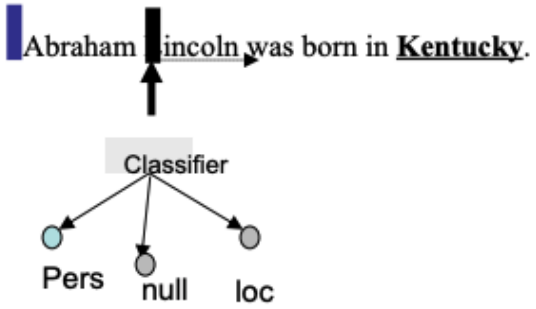
```
def read_into_feature_dict(file):
    with open(file) as le_file:
        le_dict = {}
        for line in le_file:
            line = line.strip("\n")
            if line not in le_dict:
                # other features besides frequency
                digits = sum(c.isdigit() for c in line)
                spaces = sum(c.isspace() for c in line)
                # initialize an array of [frequency, digits, spaces].
                #Frequency is initially 1
                le_dict[line] = [1,digits, spaces]

            else:
                # increment frequency if we met this before
                le_dict[line][0] = le_dict[line][0] + 1
        return le_dict
```

Two features

Dictionary, whose key is the entity, and the value is the feature array

Learning methodology: (Very) Naïve Bayes

- Input: n labeled examples in the form (x_i , y_j);
  - x_i : a feature-based descriptor based on m features $\{x_{i1},\ldots,x_{im}\}$;
  - y_j a label in Y={*1,2,...,k*},
  - *k*: number of labels.



- Task: Define a function that better maps an input *x* to output *h(x)*.
  - One way to define *h(x)* is a conditional model

$$h(x) \simeq p(y|x)$$

$$\boxed{h(x) = argmax_{y \in Y} p(y|x)}$$

For a test example x, compute the output as the most likely label

ESILV

Learning methodology: (Very) Naïve Bayes

$$h(x) \simeq p(y|x)$$

$$h(x) = argmax_{y \in Y} p(y|x)$$

- How to estimate model parameters? Apply Bayes rule

Prior probability over labels y

Probability of generating input x, given label is y

$$p(x, y) = p(y)p(x|y)$$

independent local models

$$p(x|y) = p(x_1, x_2, \ldots, x_m|y) = \prod_{j=1}^{m} p(x_j|y)$$

$$h(x) = \arg\max_{y \in Y} \tilde{p}(y)\tilde{p}(x|y)$$

- Thus, for a test token x, we can predict the label y, using
  - Where

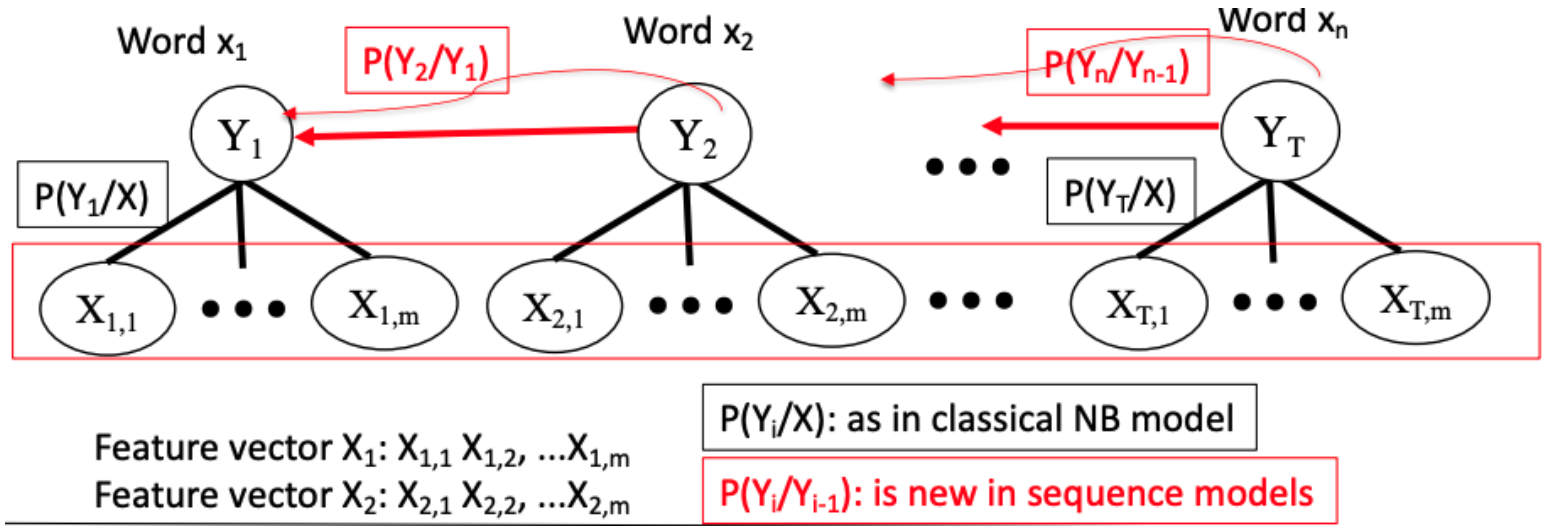
$$\tilde{p}(y) = \frac{count_y(y)}{\sum count_{y \in Y}(y)}$$

$$\tilde{p}(x|y) = \frac{count(x, y) + \alpha}{\sum_{y \in Y} count(x, y) + \alpha \times m}$$

Smoothed estimation to eliminate issues caused by certain values having 0 occurrences. Here Laplace smoothing for α=1

Learning methodology: Conditional Random Fields (CRF)

- In the NB method, texts are not classified as sequences.
- Conditional Random Fields (CRF), applies conditional decision of labelling w.r.t context, i.e., previous entity label
- Input: n labeled sequences in the form (x_i , y_j);
  - x_i: a feature-based descriptor of word w_i based on *m* features $\{x_{i1},...,x_{im}\}$;
  - y_i: a label in Y={1,2,...,k},
  - k: number of labels
- Sequence of words/tokens:



Example

Training example

X	1	2	3	4	5
	Abraham	Lincoln	born	in	Kentucky
	[Pers]	[Pers]	[Null]	[Null]	[Loc]

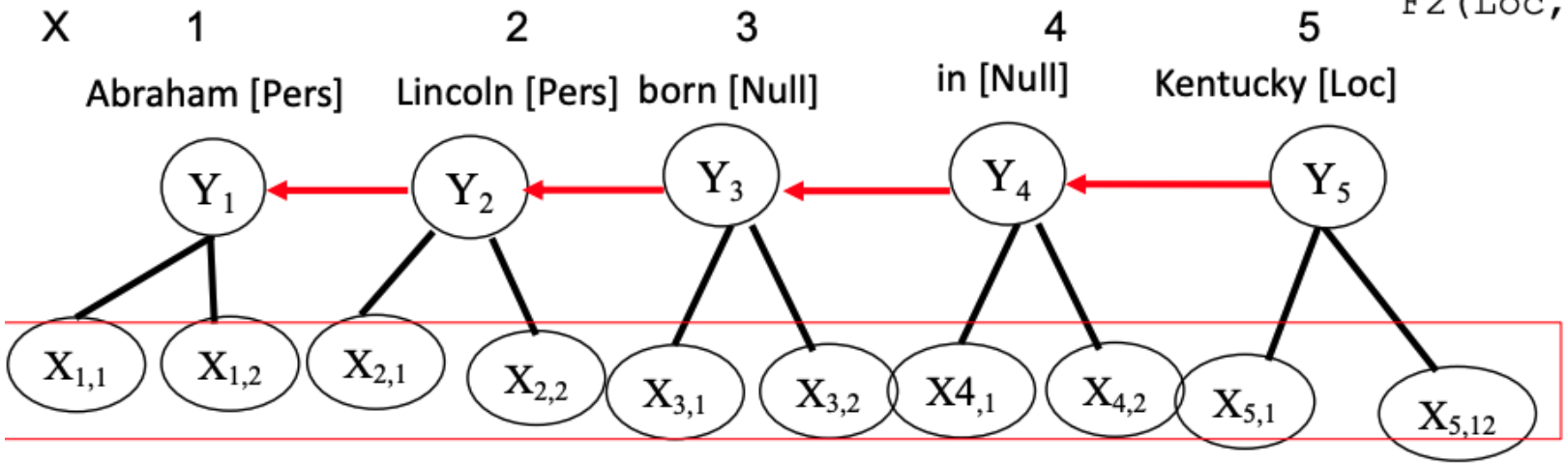
- Sequence in practice
  - x_{1j}: word is capitalised,
  - x_{2j}: word is in lexicon L.

  (Remind: x_{ij}: Feature x_i for target word j)
  - y_1:[Pers], y_2:[Loc], y_3:[Null]

Consider y_i and y_{i-1}

```
F1(pers,X,1)=1
F2(pers,X,1)=1
F1(pers,X,2)=1
F2(pers,X,2)=1
F1(null,X,3)=0
F2(null,X,3)=0
F1(null,X,4)=0
F2(null,X,4)=0
F1(Loc,X,5)=1
F2(Loc,X,5)=0
```

```
F1(pers,pers,1)=?
F2(pers,pers,1)=?
F1(pers,pers,2)=?
F2(pers,pers,2)=?
F1(null,pers,X,3)=?
F2(null,ers,X,3)=?
F1(null,null,X,4)=?
F2(null,null, X,4)=?
F1(Loc,null, X,5)=?
F2(Loc,null, X,5)=?
```



ESILV

Learning methodology: CRF (1)

- The features are same as in NB: basic or composite functions, regular expressions…
- For a given input sequence $\boldsymbol{x}=(x_1,x_2,\ldots,x_T)$(e.g., words in a sentence) and a corresponding label sequence $\boldsymbol{y}=(y_1,y_2,\ldots,y_T)$(e.g., PoS tags or entity labels), a linear-chain CRF defines the conditional probability as:

$$p(\boldsymbol{y}|\boldsymbol{x}) = \frac{1}{Z(\boldsymbol{x})}\exp\left(\sum_{t=1}^{T}\sum_{j=1}^{m}\theta_j F_j(x,y,t)\right)$$

  - It's the value of any feature function for all words of the sentence, indexed by *t*, considering label y
  - Unlike NB model, CRF models impose dependency between target successive labels, with parameters on *t*
  - **Z(x)**: The normalization factor (also called the partition function) which ensures that the probabilities sum to 1:

$$Z(\boldsymbol{x}) = \sum_{y}\exp\left(\sum_{t=1}^{T}\sum_{j=1}^{m}\theta_j F_j(x,y,t)\right)$$

ESILV

Learning methodology: CRF (2)

$$p(\boldsymbol{y}|\boldsymbol{x}) = \frac{1}{Z(\boldsymbol{x})} \exp\left(\sum_{t=1}^{T} \sum_{j=1}^{m} \theta_j F_j(x, y, t)\right)$$

- θ_j is the weight associated with the *j*-th feature function, learned during training.
- $F_j(x, y)$ is the scoring function based on the feature function on all words.

$$F_j(x, y) = \sum_{t=1}^{T} f_j(y_t, y_{t-1}, x_t)$$

- $f_j(y_t, y_{t-1}, x_t)$ is the feature extraction function, here are two examples
  - Observation feature: $f_j(y_t, x_t) = 1$ if the word x_t has a certain property and y_t is a specific tag (as in NB method). The transition of the label is not considered.
  - Transition Feature: $f_j(y_t, y_{t-1}, x_t)$=1 if y_{t-1} and y_t follow a certain pattern (e.g., if the previous label is “B-PER” and the current label is ”I-PER”, 0 otherwise). The transition of the label is considered

ESILV

CRF: training

- During training, the weights θ_j are optimized to maximize the likelihood of the training data. The objective function is usually the log-likelihood of the training set:

$$\mathcal{L}(\theta) = \sum_{i}^{N} \log p\left(y^{(i)} \middle| x^{(i)}\right) - \sum_{j=1}^{m} \frac{\theta_m^2}{2\sigma^2}$$

  - N is the number of training examples.
  - The second term is a Gaussian prior (regularization) to prevent over fitting
- Gradient-based methods, such as gradient ascent or quasi-Newton methods, are typically used to optimize the weights.

ESILV

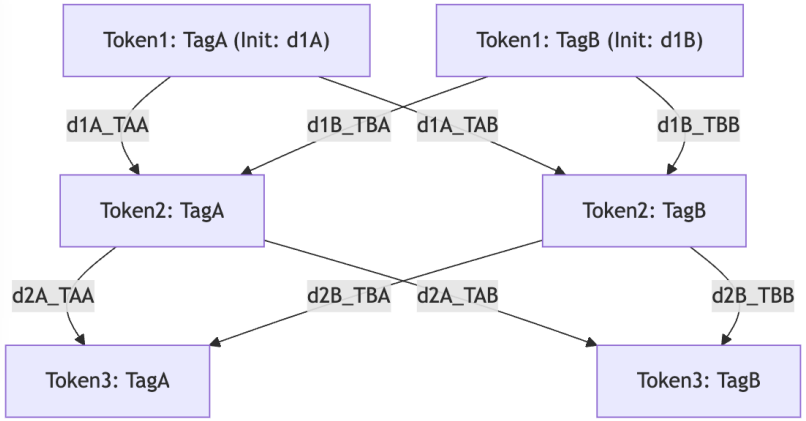
CRF: inference

$$\hat{y} = \underset{y}{\operatorname{argmax}}\, p(y|x,\theta) = \frac{\exp(\theta \times F(y,x))}{\sum_{y'} \exp(\theta \times F(y,x))}$$

- We only need to find the label y with the maximum likelihood, so the exponential inside the numerator and the denominator can be ignored

$$\hat{y} = \underset{y}{\operatorname{argmax}} \sum_{t=1}^{T} \theta \times f_j(y_t, y_{t-1}, x_t)$$

- The problem is then to find an entire sequence of labels such that the sum of the transition scores is maximized:
  - use the Viterbi algorithm
- We'll see the same issue in HMM.



Practice: NER pipeline using scikit learn

1. Feature extraction

```
def word2features(sent, i):
    word = sent[i][0]
    postag = sent[i][1]

    features = {
        'bias': 1.0,
        'word.lower()': word.lower(),
        'word[-3:]': word[-3:],
        'word[-2:]': word[-2:],
        'word.isupper()': word.isupper(),
        'word.istitle()': word.istitle(),
        'word.isdigit()': word.isdigit(),
        'postag': postag,
        'postag[:2]': postag[:2],
    }
    if i > 0:
        word1 = sent[i-1][0]
        postag1 = sent[i-1][1]
        features.update({
            '-1:word.lower()': word1.lower(),
            '-1:word.istitle()': word1.istitle(),
            '-1:word.isupper()': word1.isupper(),
            '-1:postag': postag1,
            '-1:postag[:2]': postag1[:2],
        })
    else:
        features['BOS'] = True
    if i < len(sent)-1:
        word1 = sent[i+1][0]
        postag1 = sent[i+1][1]
        features.update({
            '+1:word.lower()': word1.lower(),
            '+1:word.istitle()': word1.istitle(),
            '+1:word.isupper()': word1.isupper(),
            '+1:postag': postag1,
            '+1:postag[:2]': postag1[:2],
        })
```

Practice: NER pipeline using scikit learn

- 2. split train and test data

```
X = [sent2features(s) for s in sentences]
y = [sent2labels(s) for s in sentences]
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.33, random_state=0)
```

- 3. Train a CRF model

```
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,
    max_iterations=100,
    all_possible_transitions=True
)
crf.fit(X_train, y_train)
```

- To see in lab session

ESILV

In summary

- Advantages
  - Naive Bayes methods: naive but effective
  - Experimental results show that CRF have superior accuracy
- Limitations
  - CRFs are much slower to train and do not scale as well to large amounts of training data
- Traditional ML-based classifiers for NER take a set of features as input describing raw text units **U** and outout a prediction of the entity class $\mathbf{C_U}$
- Traditional ML-based classifiers for NER require:
  - Design features to represent raw text units. Combine features for larger units (feeture-engineering)
  - Select a model to combine these features and make a prediction
  - Train the model on a human-labelled corpus
  - Test on unlabeled raw texts

ESILV

Approaches for NER

- Rule-based
  - Manual/hand-written patterns
- Unsupervised learning
  - Mostly clustering techniques
  - Discriminative models
- Feature-based supervised learning
  - Naive Bayes, Hidden Marko Models (HMM), Conditional Random Field (CRF), Maximum-Entropy Markov Models (MEMMs)
- **(Deep) Neural - based learning**
  - Distributed representation of inputs, deep encoding of context, tags

ESILV

Why deep learning for NER?

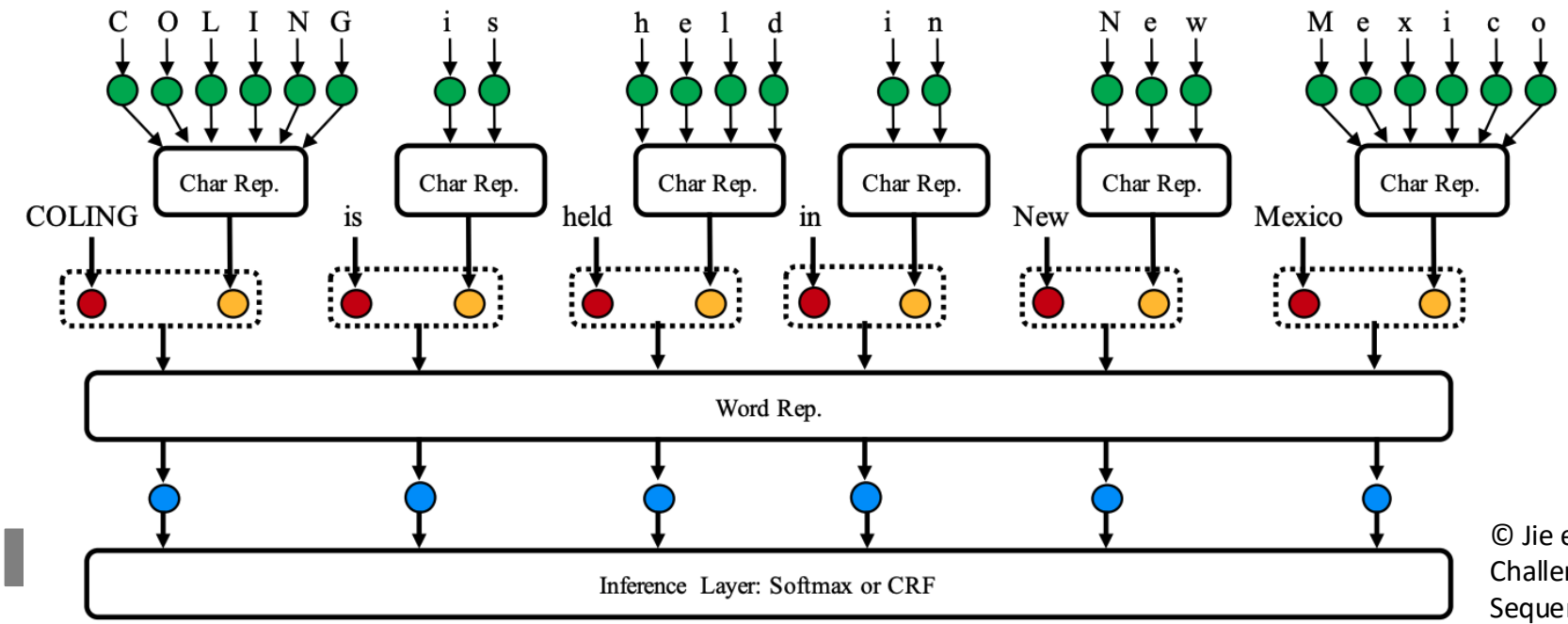
- Learn nonlinear mapping between input text and output labels
  - Naive Bayes, Linear CRF learn linear combinations
- No requirement for feature engineering
  - Is model ineffective or features are not relevant in NB, CRF?
  - DL learns mappings from raw data, not features
- Learning in an end-to-end framework
  - Learn task-based representations of labels: suited representation of texts for NER, RE, ..

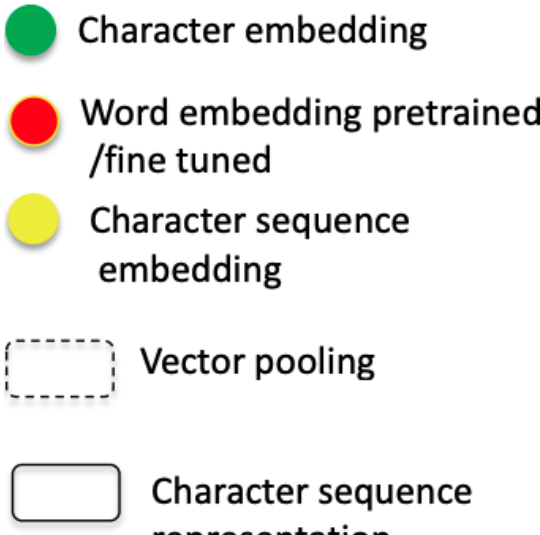
- Approaches:
  - Distributed (contextless) representations of input
    - Use **word embeddings** to capture semantic and syntactic properties of words
    - Word2vec, Glove, etc...
  - Sequence-based models of input
    - Consider word in the whole sequence of words (sentence)
    - LSTM, RNN, Pointer architectures mostly
  - Stacked self-attention within context
    - Transformers

ESILV

Distributed representations of input

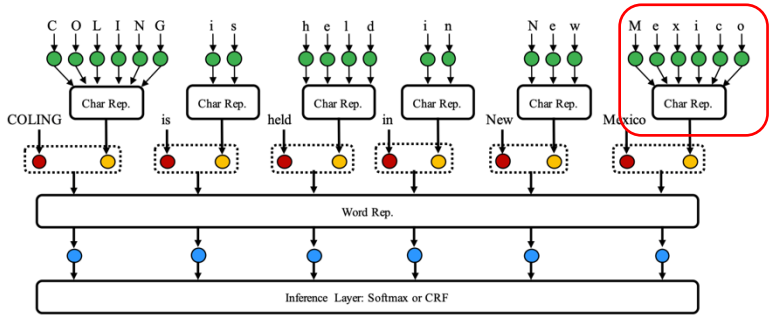
- Use **embeddings** at input (refer to the tutorial)
  - Character embeddings, word embeddings and pool the embeddings
- Output as a classifier prediction
  - Traditional linear combination of inputs (e.g., CRF)
  - Neural inference based on embeddings (e.g., perceptron)



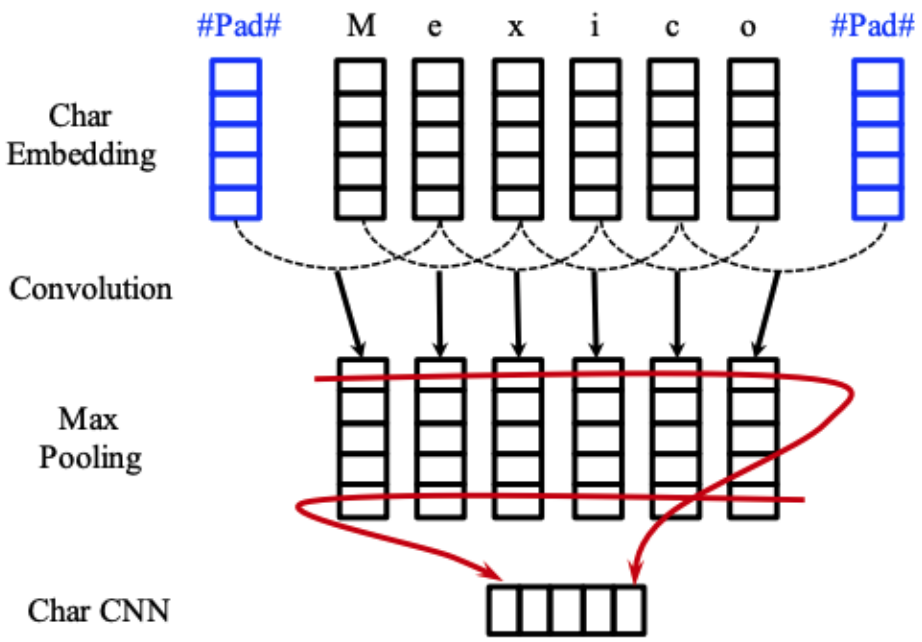


© Jie et al. 2019, COLING, 2018: Design Challenges and Misconceptions in Neural Sequence Labeling

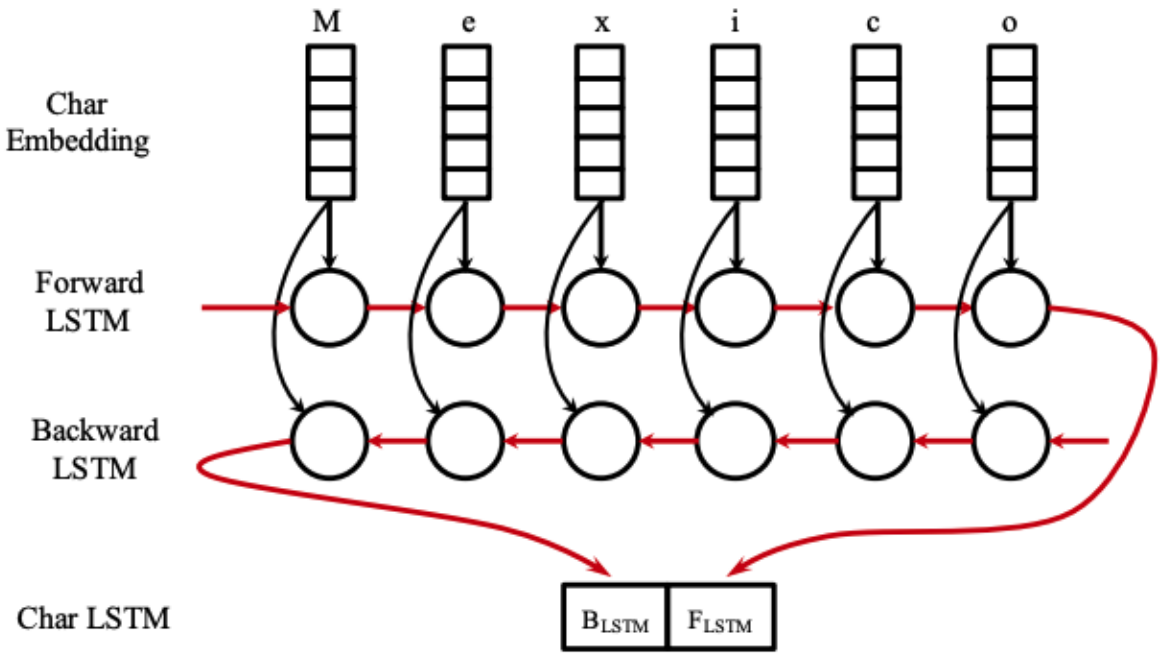
Neural character sequence representations



By using character embedding and word embedding, the model learns Important factor for neural NER either for characters and words representation

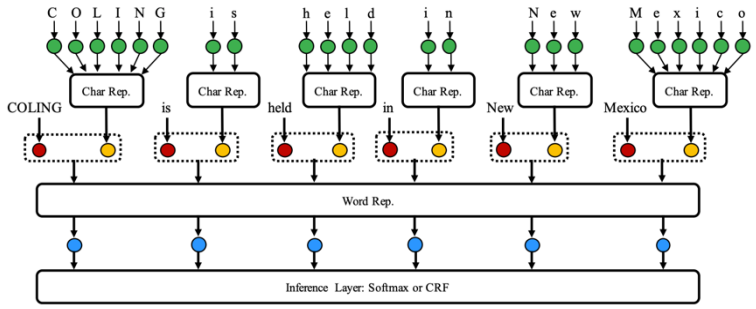


(a) Character CNN.

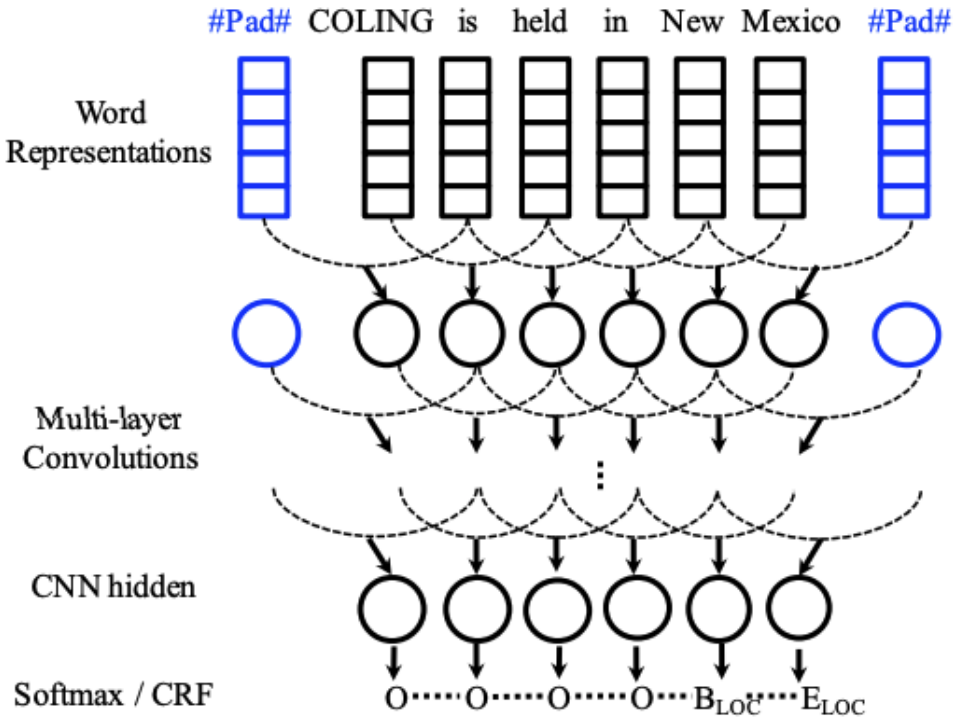


(b) Character LSTM.

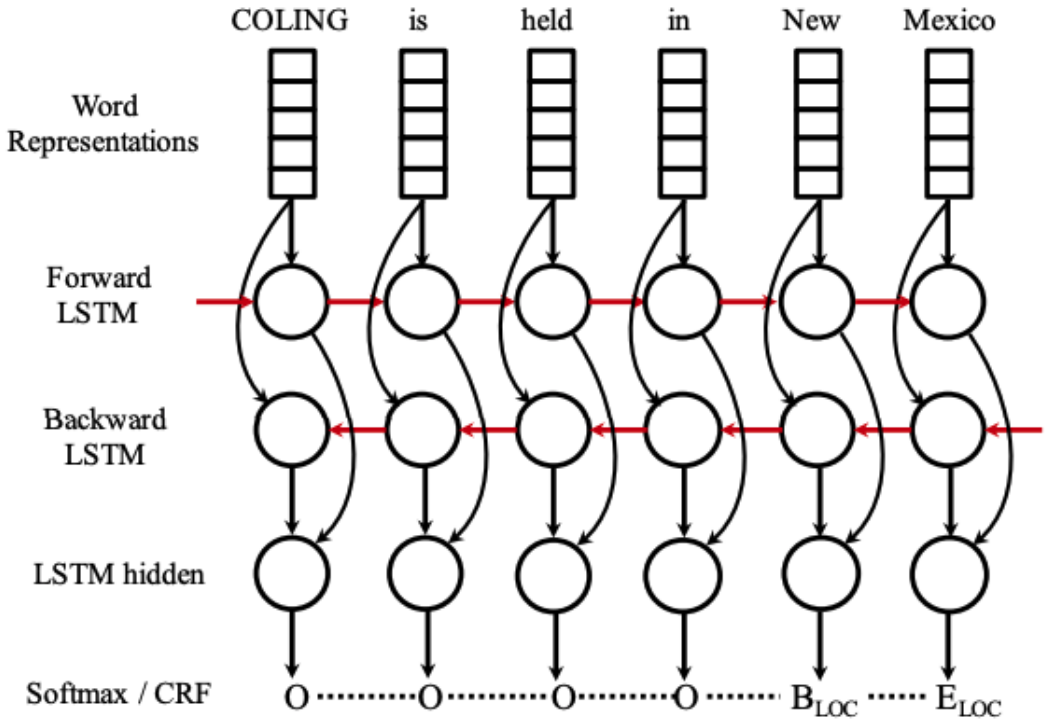
Neural word sequence representations



words are represented by embeddings



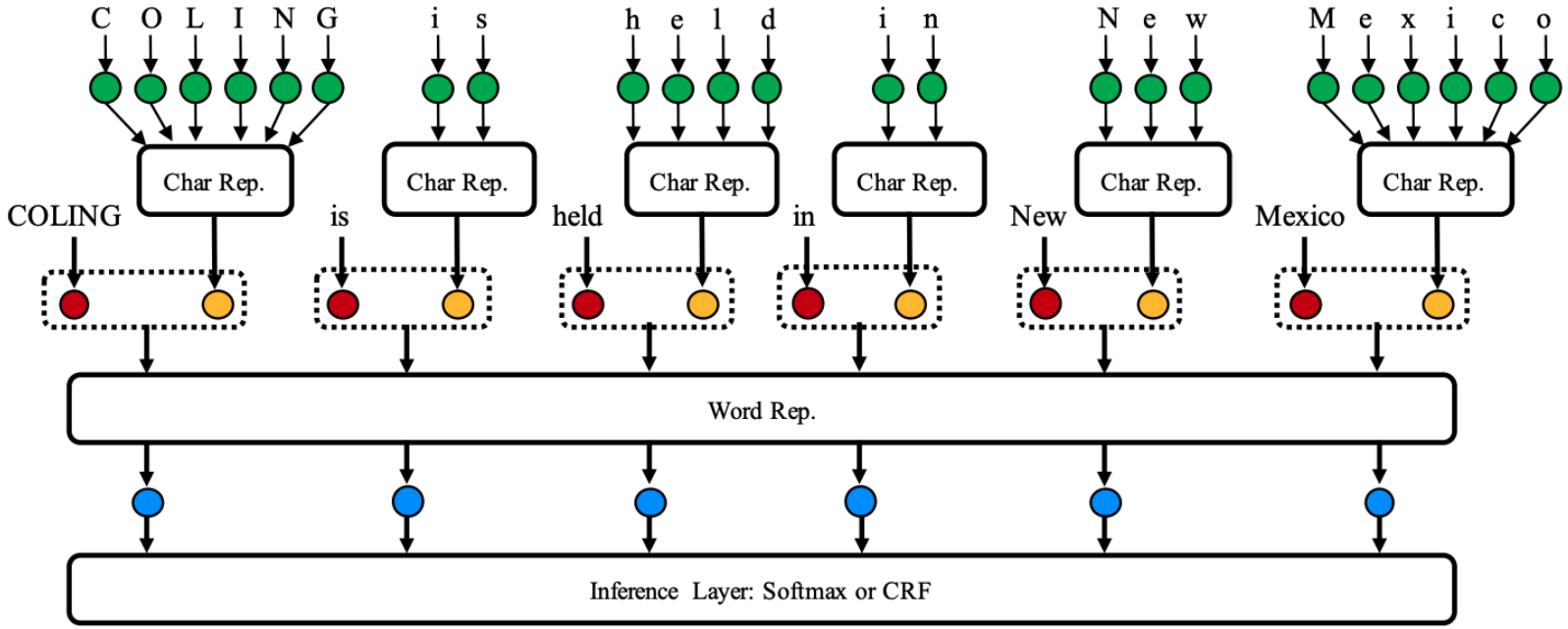
(a) Word CNN.



(b) Word LSTM.

Inference layer

- The inference layer takes the extracted word sequence representations as features and assigns labels to the word sequence.
- Use either softmax or CRF as inference layer



Evaluation of link NER

- Measure how much the golden standard annotations map the output of a NER system different scenarios. Build a 2-by-2 contingency matrix.

TP: True Positive FN: False Negative	FP: False Positive TN: True Negative	Gold standard	
		Correct	Not correct
NER system	Selected	TP	FP
	Not selected	FN	TN

- Main measures

Precision (P): proportion of selected items that are correct

$$P = \frac{TP}{TP + FP}$$

Recall (R): proportion of corrrect items that are selected

$$R = \frac{TP}{TP + FN}$$

F_measure: a combined

$$F = \frac{1}{\theta \times \frac{1}{P} + (1 - \theta) \times \frac{1}{R}}$$

Results and conclusions

- Character embeddings are more effective than raw character representations
  - No significant differences between CNN and LSTM based representations
- Word embeddings are more effective than raw words
  - LSTM-based representations are more effective than CNN-based representations
  - Neural inference based on embeddings (eg. perceptron)
- Softmax-layer is more effective than CRF for prediction output

Results (F1-score)			NER			
			WLSTM+CRF	WLSTM	WCNN+CRF	WCNN
Nochar	Literature		90.10 (H-15)* 90.20 (L-16) 90.43 (S-17)*	87.00 (M-16) 89.34 (S-17)*	89.59 (C-11)* 90.54 (S-17)*	89.97 (S-17)*
	Ours	Max	89.45	88.57	88.90	88.56
		Mean±std	89.31±0.10	88.49±0.17	88.65±0.20	88.50±0.05
CLSTM	Literature		90.94 (L-16) 91.20 (Y-17)‡	89.15 (L-16)	–	–
	Ours	Max	91.20	90.84	90.70	90.46
		Mean±std	91.08±0.08	90.77±0.06	90.48±0.23	90.28±0.30
CCNN	Literature		90.91±0.20 (C-16) 91.21 (M-16) 90.87±0.13 (P-17)	89.36 (M-16)	–	–
	Ours	Max	91.35	90.73	90.43	90.51
		Mean±std	91.11±0.21	90.60±0.11	90.28±0.09	90.26±0.19

ESILV

Transformer based

- The most used one today. To see in next session.

ESILV

Datesets

Corpus	Year	Text source	#Entity categories	URL
MUC-6	1995	Wall Street Journal	7	https://catalog.ldc.upenn.edu/LDC2003T13
CoNLL03	2003	Reuters News	4	https://www.clips.uantwerpen.be/conll2003/ner/
ACE	2000- 2008	Transcripts, news	7	https://www.ldc.upenn.edu/collaborations/past-projects/ace
OntoNotes	2007- 2012	Magzines, news, web	18	https://catalog.ldc.upenn.edu/LDC2013T19
WikiGold	2009	Wikipedia	4	https://figshare.com/articles/Learning_multilingual_name d_entity _recognition_from_Wikipedia/5462500
WiNER	2012	Wikipedia	4	http://rali.iro.umontreal.ca/rali/en/winer-wikipedia-for-ner
BC5CDR	2015	PubMed	3	http://bioc.sourceforge.net/
DFKI	2018	Business news and social media	7	https://dfki-lt-re-group.bitbucket.io/product-corpus/

Relation Extraction (RE)

- Relation extraction (RE) is the process of identifying and classifying semantic relationships between NE in text.
  - Example: From "Barack Obama was born in Hawaii", extract the relation **BornIn(Barack Obama, Hawaii).**
- Both RE and NER are essentially annotation process.
  - They share common learning methodologies and feature extraction.
    - We can consider the relations as specific annotation labels
  - Differences:
    - RE often requires additional features that capture the connection between pairs of entities.
    - The output of RE are structured relations, in form of triples, while NER outputs only a sequence of lables

ESILV

RE Examples (Marie Curie discovered Polonium)

- Rule-Based (Pattern-Based) Extraction
  - **Method:** Hand-crafted linguistic pattern.
  - **Example Rule:** X discovered Y → discovered(X, Y)
  - Output: (discovered, Marie Curie, polonium)
- Feature-Based Supervised Machine Learning
  - **Method:** Train a classifier using engineered features.
  - **Input Features (Simplified)**
    - Entity types: Person, ChemicalElement
    - Dependency path: nsubj(discovered), dobj(discovered)
    - Token distance: 2

ESILV

RE Examples (Marie Curie discovered Polonium)

- CNN / RNN-Based Neural Models
  - **Method:** Sentence encoded as a sequence of word embeddings.
    - [Marie] [Curie] [discovered] [polonium] [in] [1898]
  - **Model Output**
    - Relation: discovered
    - Confidence: 0.91
- Transformer-Based Relation Extraction (BERT-style) (to see in next session)
  - **Method:** Contextual encoding with entity markers.
  - **Input:** [CLS] <e1>Marie Curie</e1> discovered <e2>polonium</e2> in 1898. [SEP]
  - **Model Output:** Relation = discovered

ESILV

Next Lecture

- Transformer in IR.

ESILV